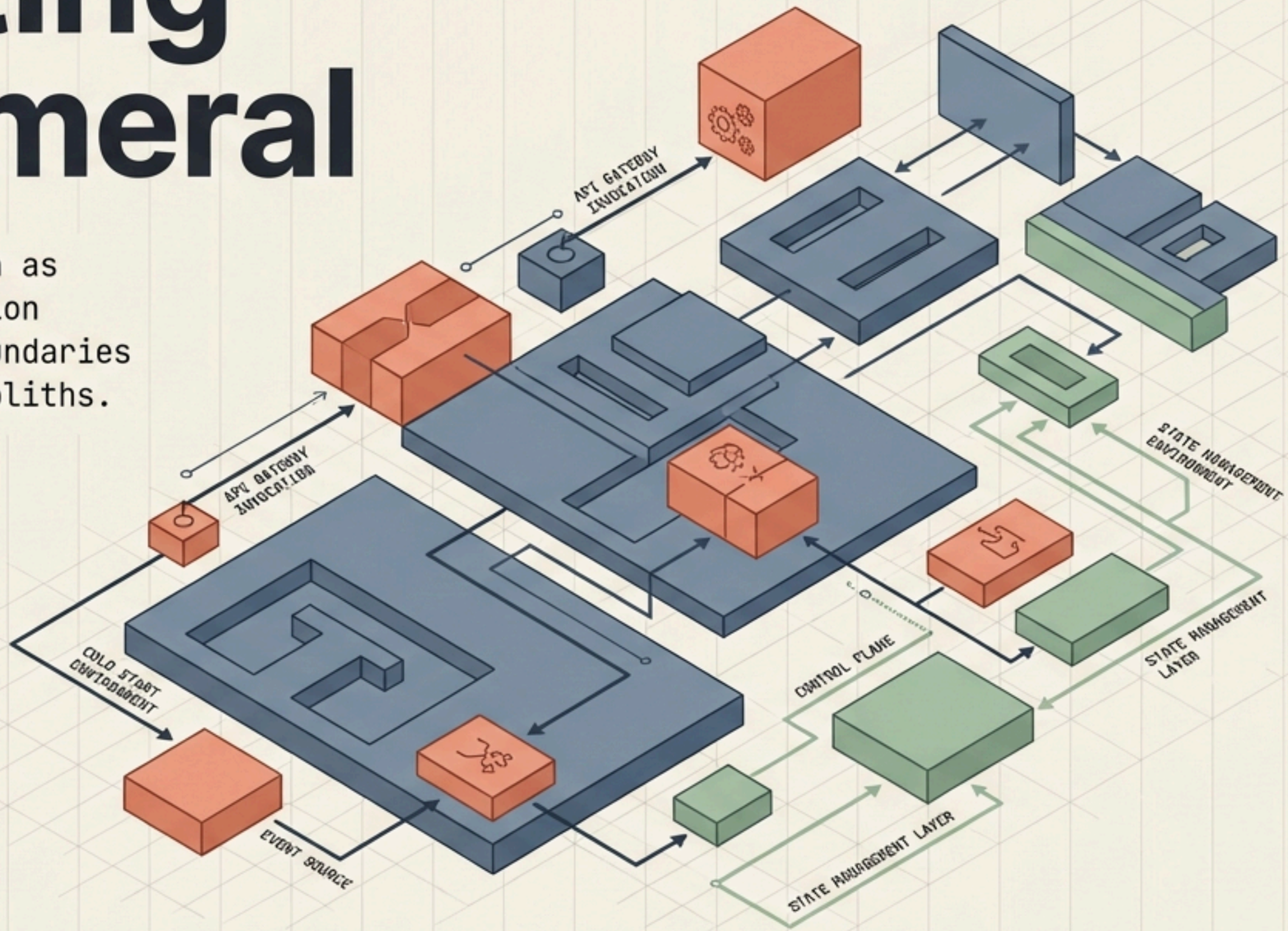
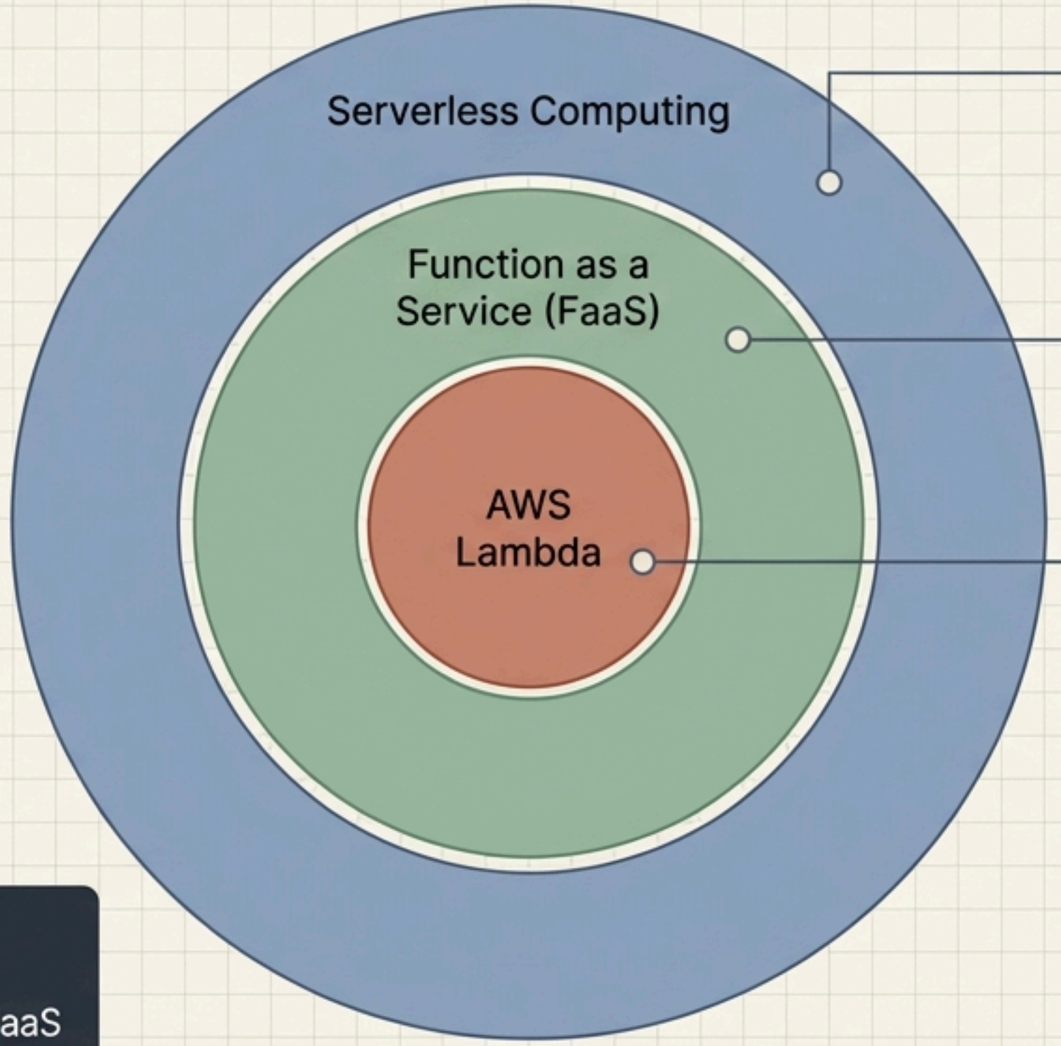
Architecting the Ephemeral

A technical distillation of Function as a Service (FaaS), AWS Lambda execution mechanics, and the architectural boundaries required to prevent distributed monoliths.



1. P.Yashwanth-2320090012
2. T.Divyesh-2320030080
3. A.Kowshik-2320030121
4. A.Prashath sai-2320030320

AWS Lambda operates at the core of the serverless ecosystem



A broad ecosystem abstracting hardware and software provisioning, including Managed Services and BaaS.

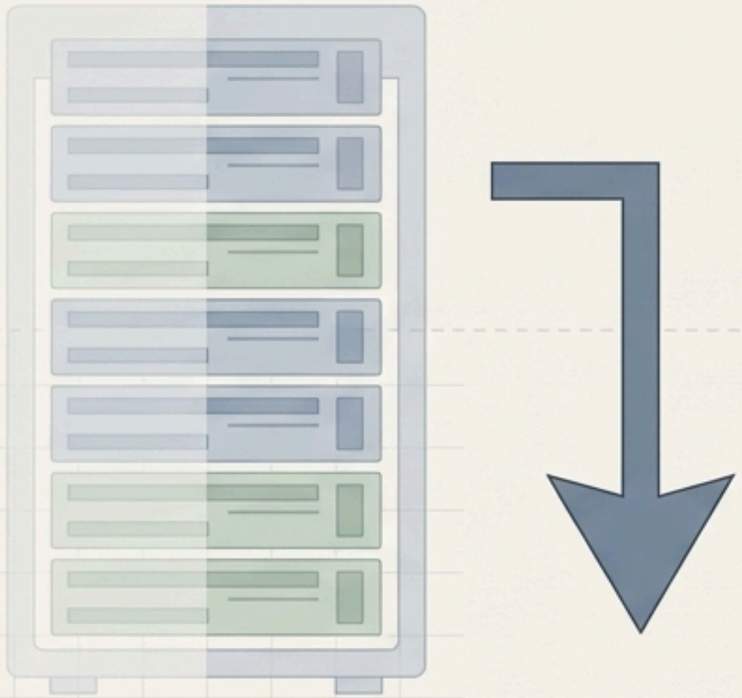
A subset focusing on building and managing microservices with event-driven execution.

Amazon's specific implementation, launched in November 2014, running code on demand without persistent infrastructure.

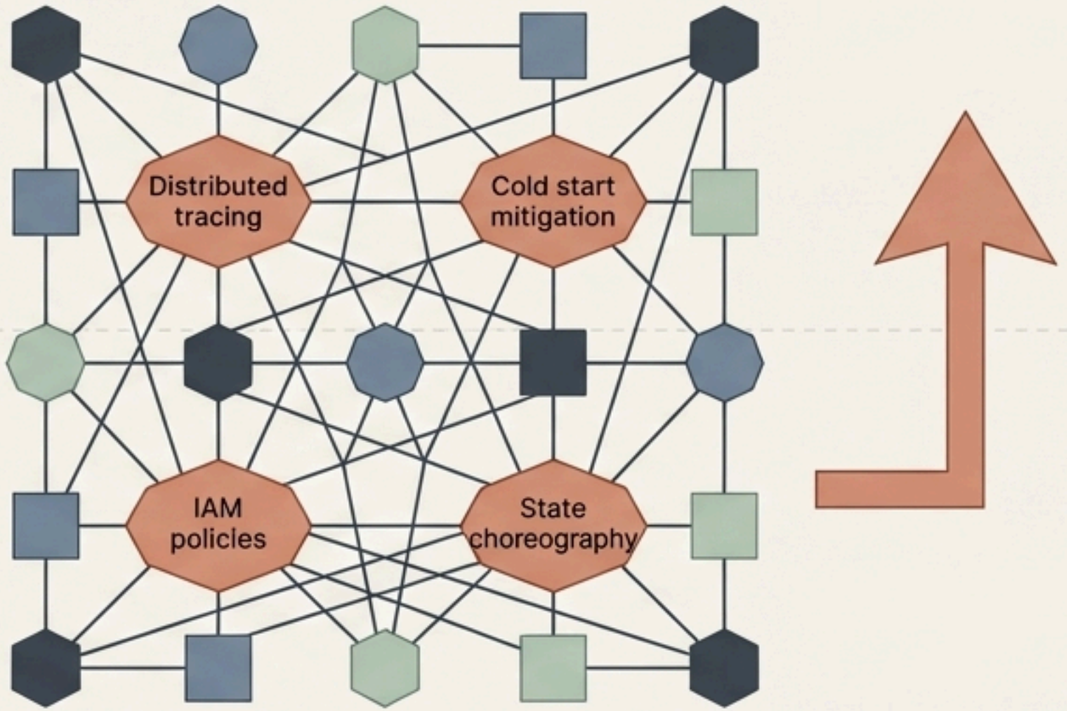
Serverless is a spectrum. While the ecosystem includes everything from managed databases to API gateways, FaaS provides the programmable execution layer.

Serverless shifts complexity rather than eliminating it

Operational Burden

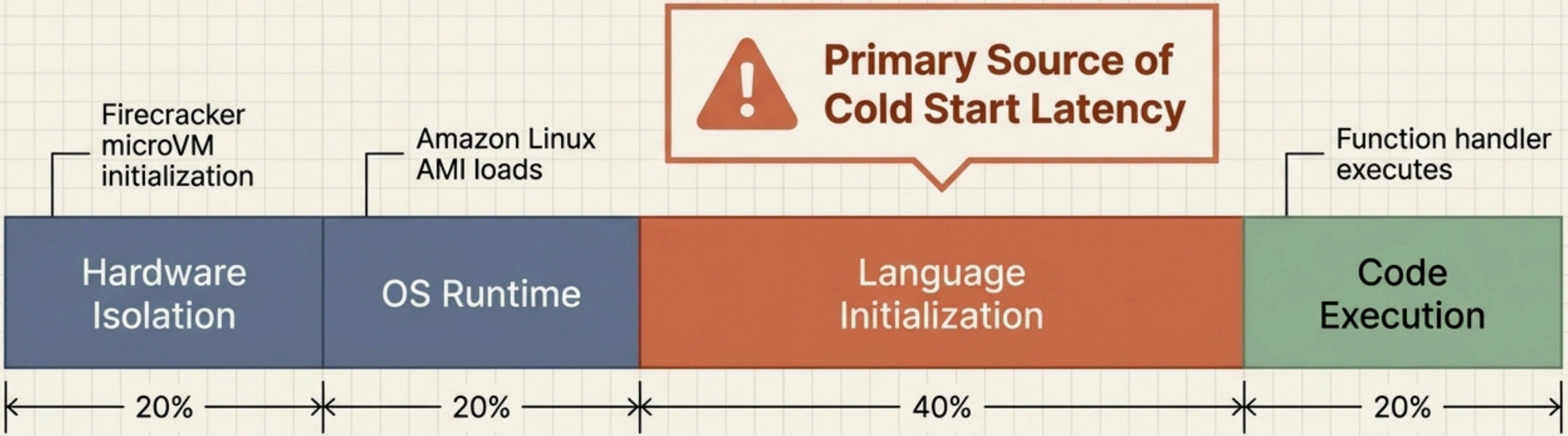


Architectural Burden



By abstracting the server, AWS Lambda transfers the weight of system reliability from the operations team directly to the development team. The relationship between component granularity and management difficulty is not linear—hyper-fragmentation creates new friction.

Cold starts occur within the microsecond boot sequence of Firecracker microVMs

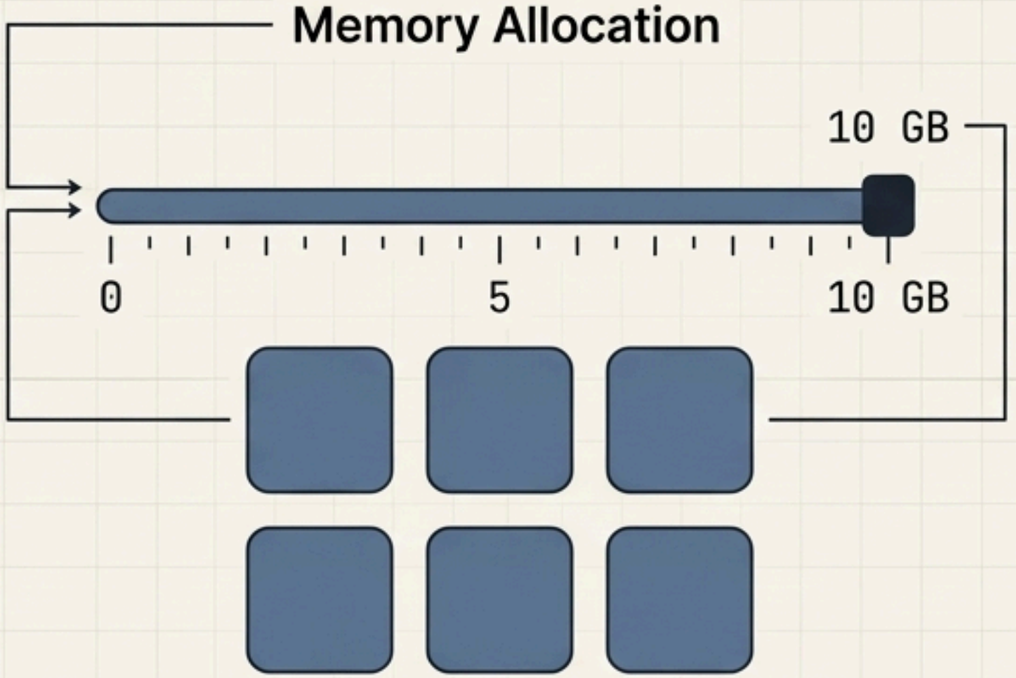


Firecracker provides hardware-virtualization-based isolation, launching in milliseconds to achieve near-bare-metal performance. However, runtime dependencies introduce critical initialization delays before the first execution.

Runtime choices dictate cold start latency and execution predictability

Language	Compilation Type	Cold Start Latency Profile	Optimization Workarounds
Rust / Go	AOT (Ahead-of-Time) / Native Static Binaries	**Ultra-low**	None required. **Rust** achieves lowest latency; **Go** has minimal **garbage collection** overhead. Ideal for short-lived, predictable invocations.
Java / C# (.NET)	**JIT** (Just-In-Time) / Managed Runtime	High initial overhead	AWS Lambda **SnapStart** pre-warms and snapshots execution state for **Java 11/17**. .NET 7 and 8 utilize Native AOT compilation to precompile code and bypass the CLR overhead.

Vertical scaling unlocks parallel vCPUs without increasing single-thread speed



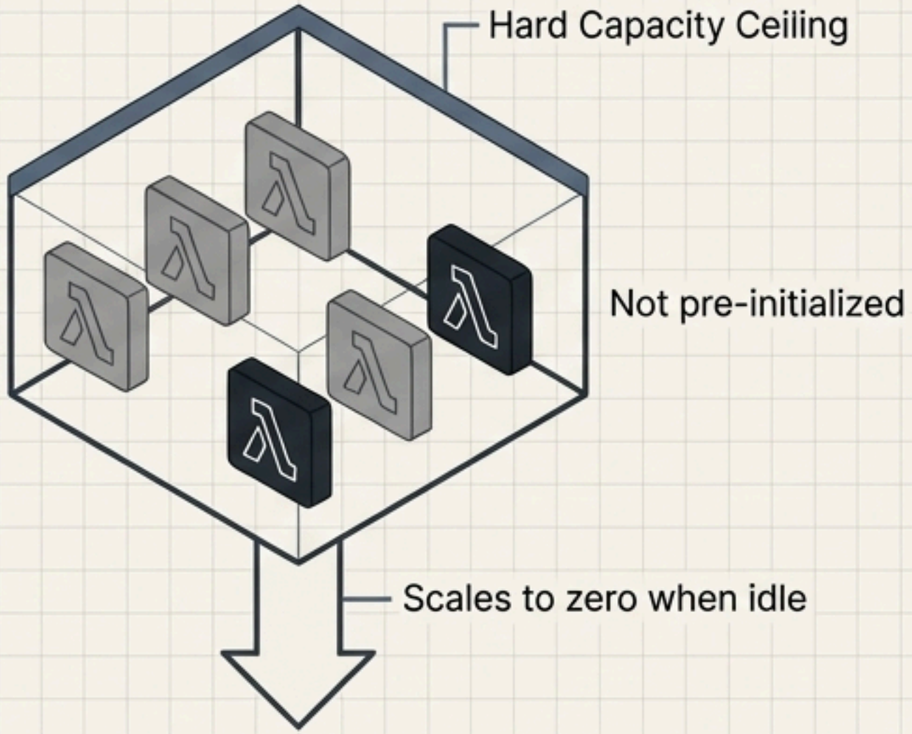
Unlocks up to 6 parallel vCPU cores

Adding memory does not increase per-core clock speed.

When allocated a single vCPU, multiple threads merely context-switch on the same core. Additional memory enables true parallel execution across multiple vCPUs, making Lambda highly effective for I/O-bound or horizontally scalable workloads, but inherently constrained for high-performance single-thread compuread computation.

Concurrency models dictate environment initialization and scale-to-zero behavior

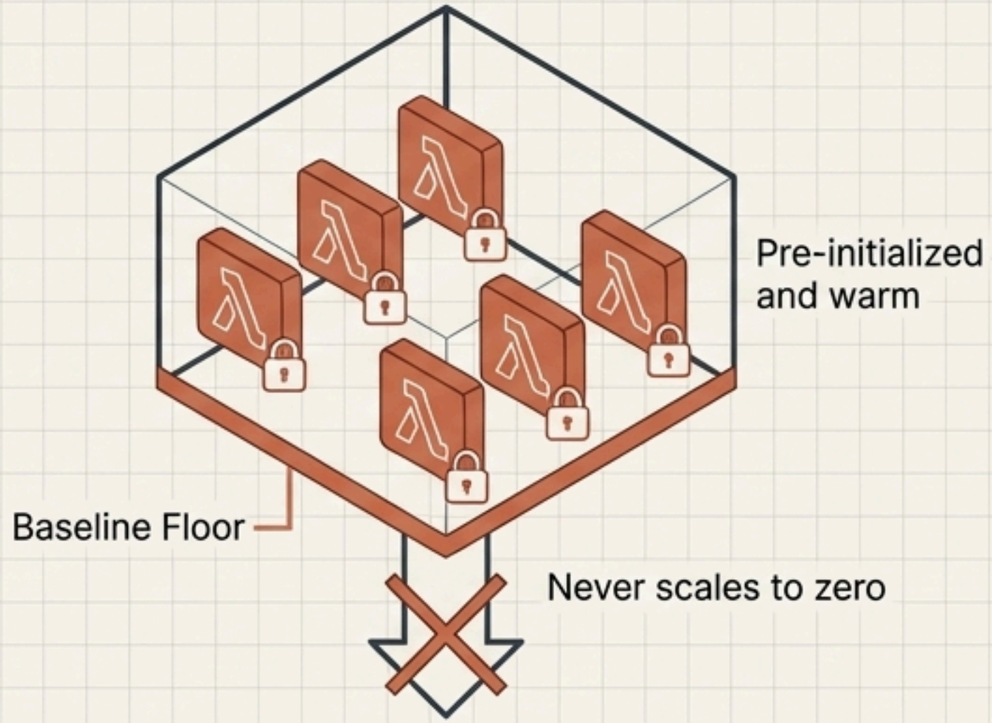
Reserved Concurrency



Use Case

Cost control and protecting downstream systems from overwhelming traffic.

Provisioned Concurrency



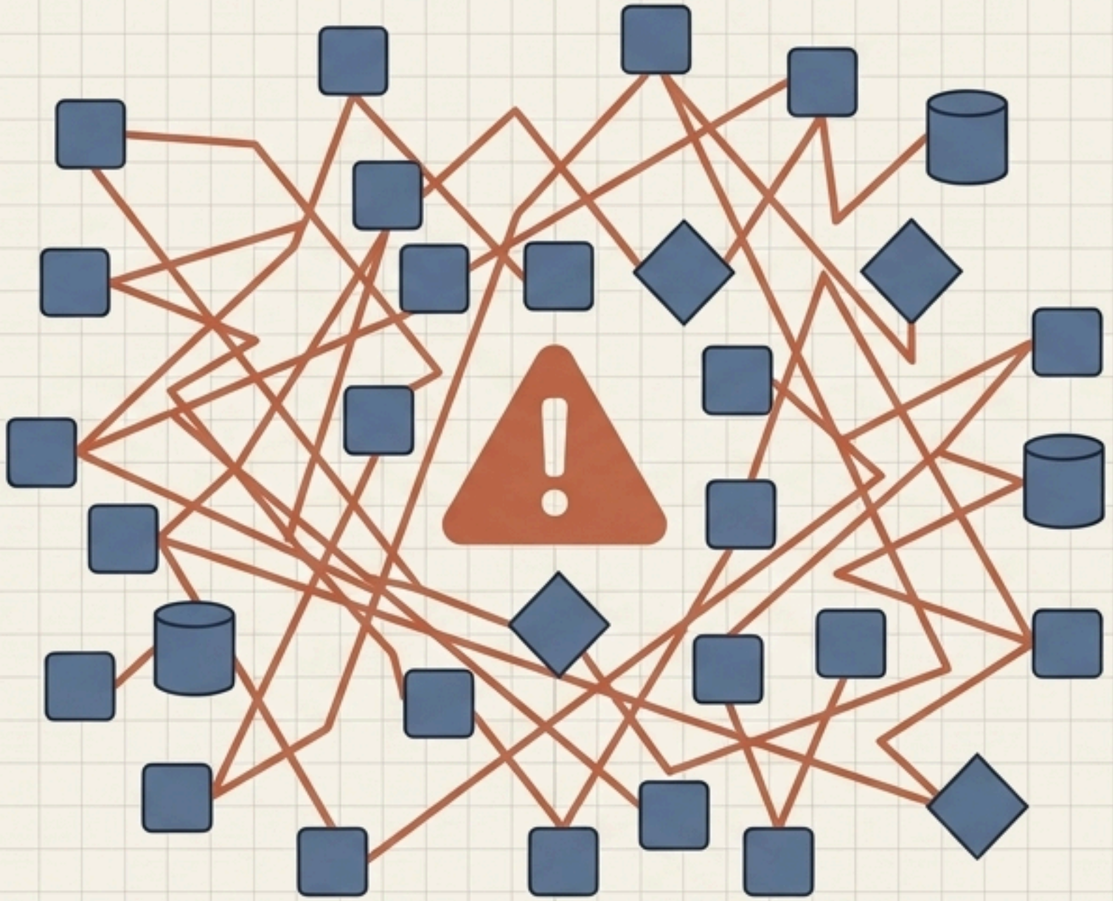
Use Case

Eliminating cold starts for latency-sensitive APIs by paying a premium for warm environments.

Treating functions like traditional code creates distributed monoliths

Grain of Sand Anti-pattern

Creating excessively small components, resulting in crushing crushing operational overhead and performance drag.



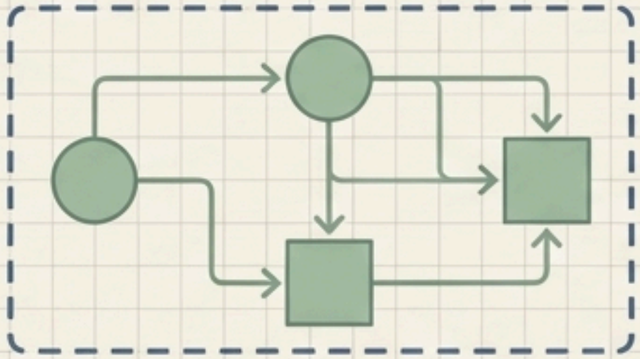
Spaghetti Topology

Lambda Pinball Anti-pattern

Functions excessively invoking each other invoking each other in fragmented chains, destroying observability, complicating testing, and adding cumulative latency.

Clear interface boundaries prevent fragmented function chaining

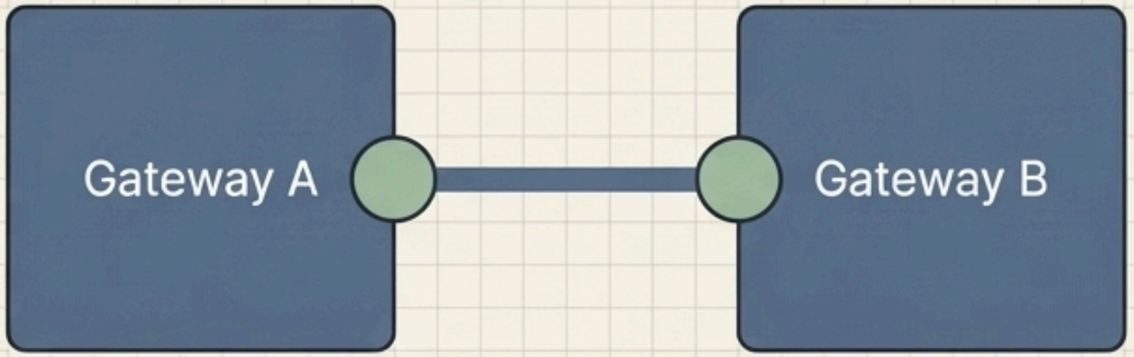
Public Interfaces



Technically accessible endpoints, methods, or triggers used internally within a cohesive group.

Constraint: No formal stability guarantees.

Published Interfaces



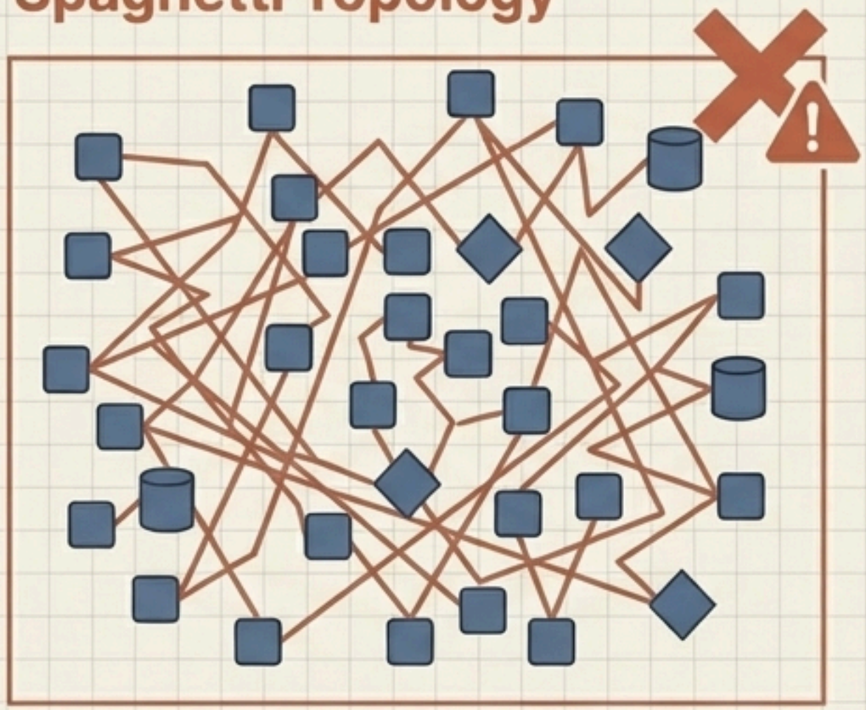
Explicit stability contracts used to communicate across system boundaries.

Requirements: Formal versioning, thorough documentation, backward compatibility, and defined deprecation policies.

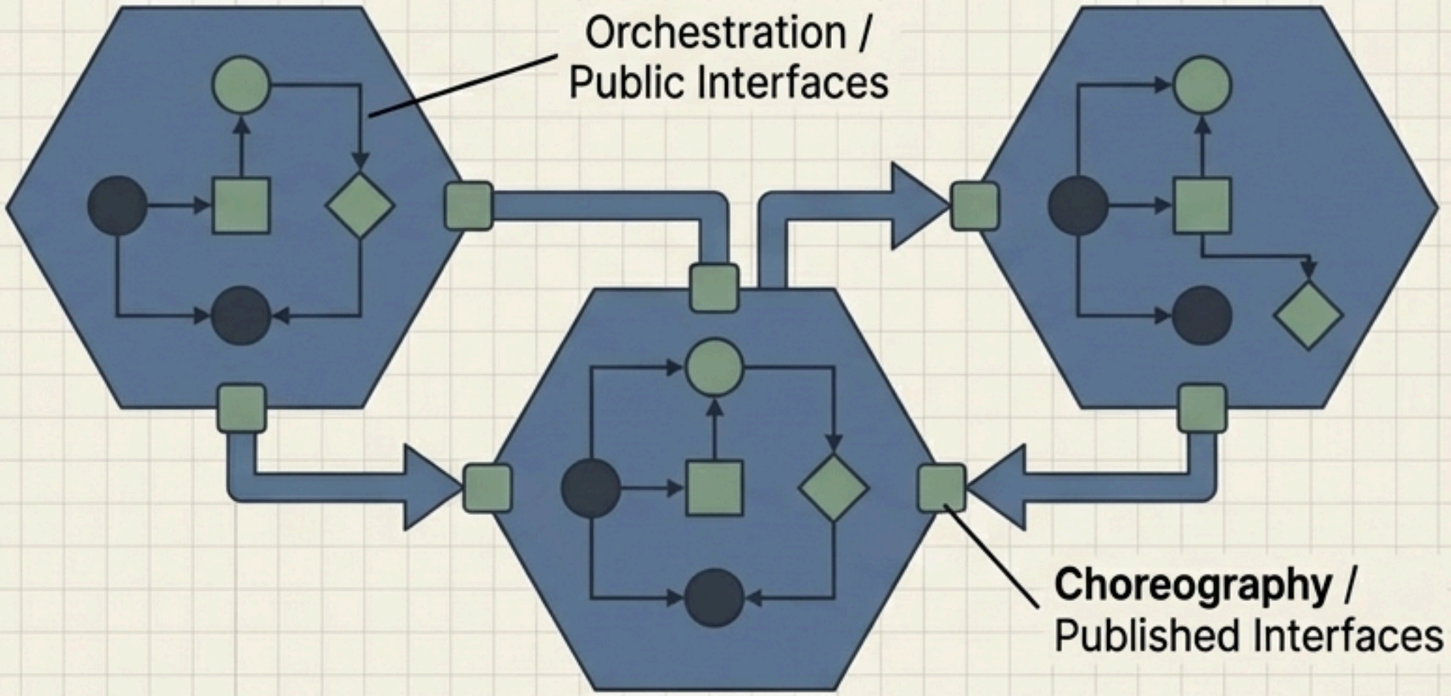
To dismantle a distributed monolith, serverless components must be organized into cohesive groups. Internal 'Public' interfaces handle local communication, while rigid 'Published' interfaces dictate interactions between bounded contexts.

Topologies of control: Orchestrate locally, choreograph globally

Spaghetti Topology

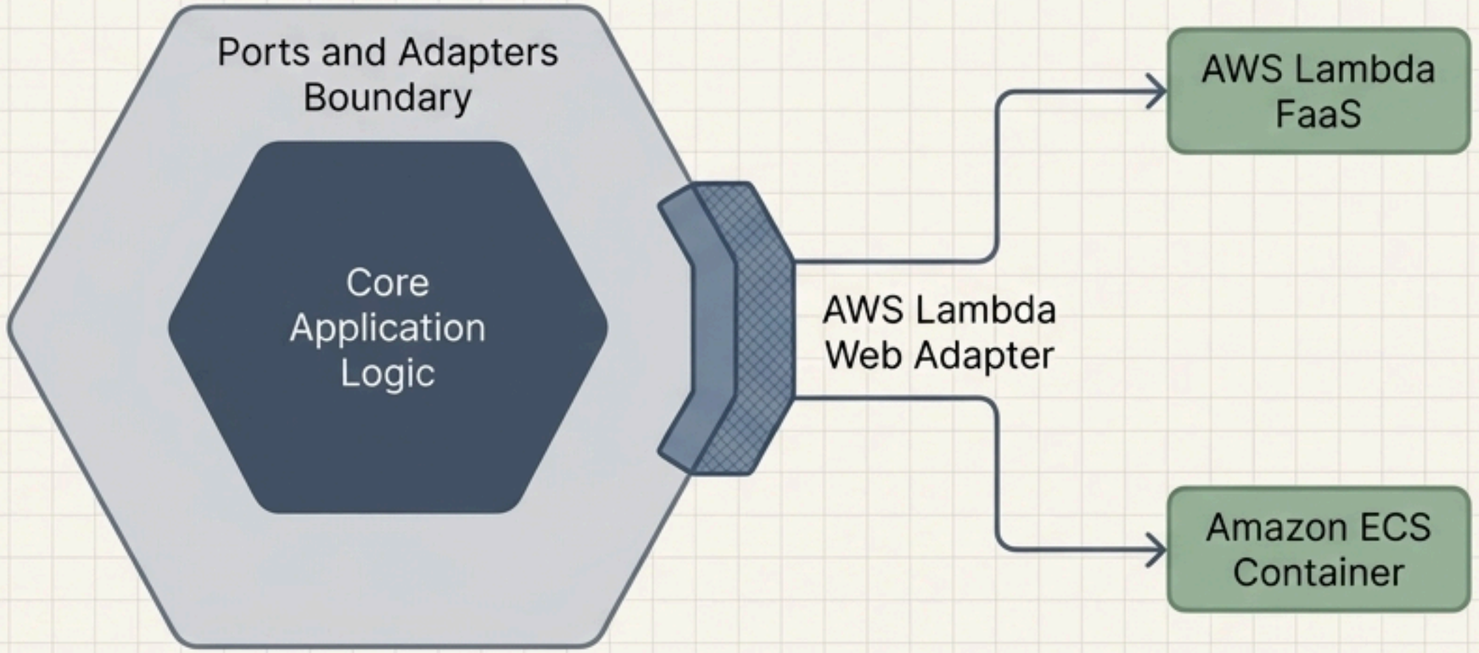


Bounded Contexts



Architecture must distinguish between **orchestration** and **choreography**. Keep ephemeral resources highly cohesive. Complex integrations and shared resources (like RDS clusters) belong in isolated deployments to preserve bounded contexts.

Portability requires intentional architectural design to avoid service lock-in



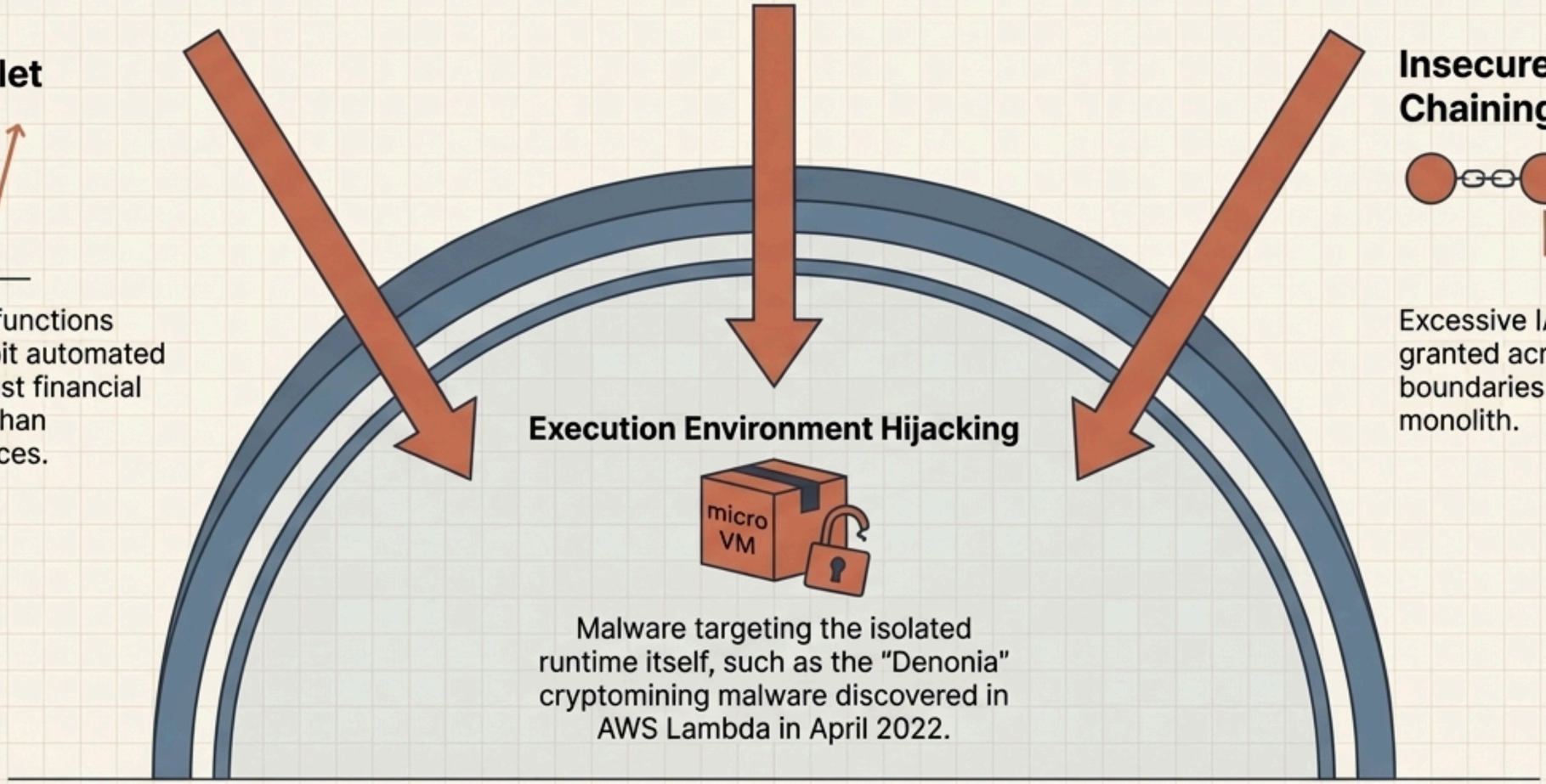
FaaS environments natively trigger services from the same provider, making migration virtually impossible by default.

Tools like the AWS Lambda Web Adapter enable developers to use monolithic design patterns (like familiar web frameworks) that can run on both Lambda and containerized services, though this sacrifices granular access controls.

Ephemeral infrastructure introduces distinct attack vectors

Denial of Wallet

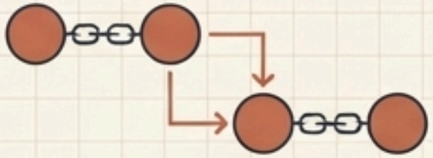
Attackers trigger functions endlessly to exploit automated scaling and exhaust financial resources rather than computing resources.



Execution Environment Hijacking

Malware targeting the isolated runtime itself, such as the "Denonia" cryptomining malware discovered in AWS Lambda in April 2022.

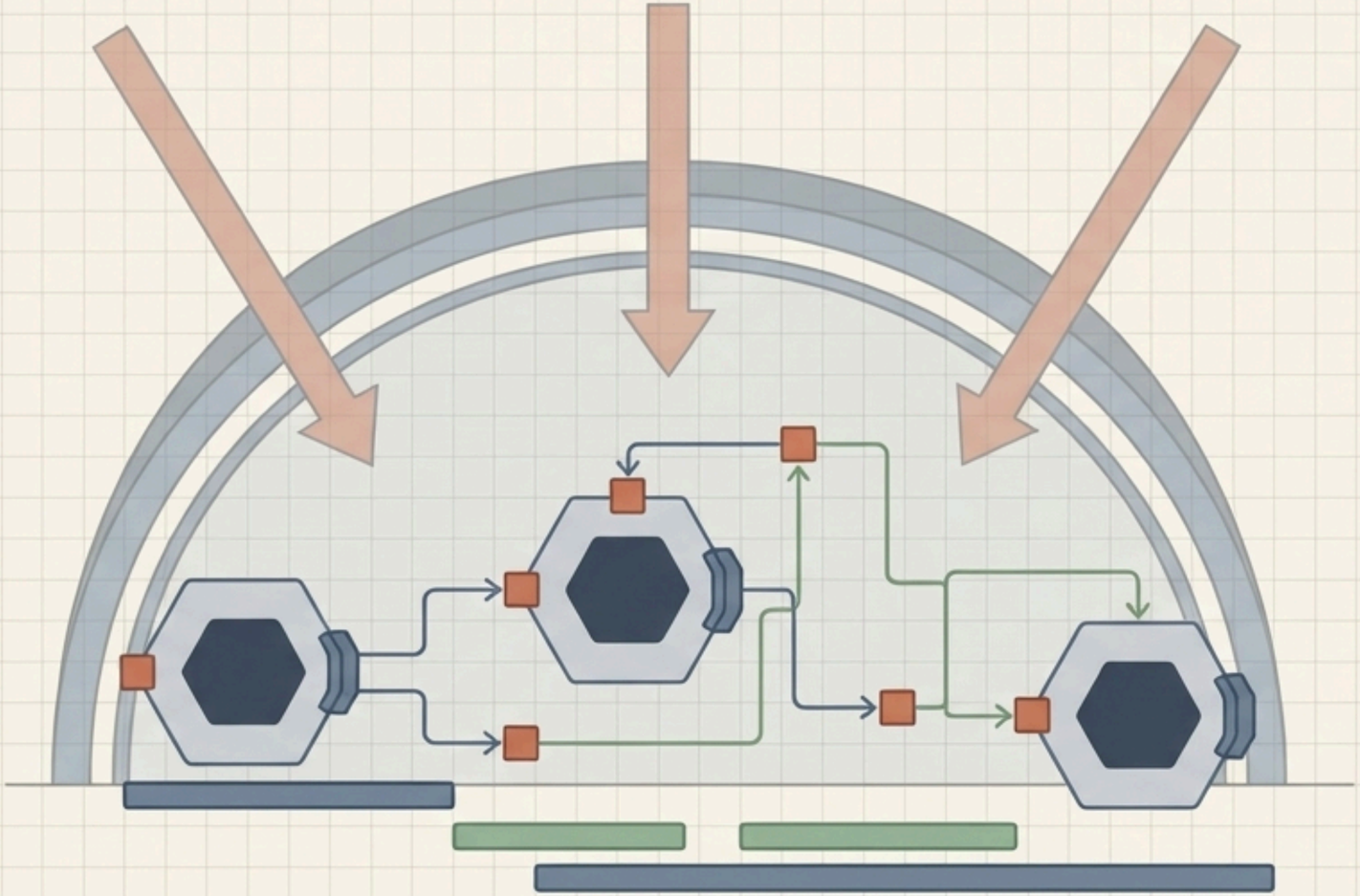
Insecure Function Chaining



Excessive IAM permissions granted across poorly defined boundaries in a distributed monolith.

Serverless security requires a shift in mindset. Traditional server vulnerabilities are abstracted away, but the OWASP Serverless Top 10 highlights that automated scaling uniquely weaponizes application layer flaws into severe financial and operational risks.

The Serverless Mindset: Architecture over execution



Inter text with `JetBrains Mono` highlights: **Serverless computing is not a technology; it is an operating model.** Success in AWS Lambda is not achieved by writing the fastest function on a Firecracker microVM. It is achieved by **drawing the right Bounded Contexts**, designing Published Interfaces, and realizing that mastery of FaaS is ultimately **the mastery of architectural choreography**.